

QUICK SORT — Complete Learning Notes

Method used throughout: first element as pivot + Hoare partition scheme.

Learning goal: understand the workflow, pointer movement, partitioning, recursion, code, and complexity — not memorize the code.

1. The Core Idea

Quick Sort is a **divide-and-conquer** sorting algorithm. Its central operation is **partitioning**: choose a pivot and rearrange the current section so smaller values are on the left side and larger values are on the right side.

Mental model:

```
Quick Sort
↓
Choose pivot
↓
Partition
↓
Split into two smaller sections
↓
Quick Sort the left section
Quick Sort the right section
↓
Eventually sorted
```

2. Our Rules

Rule	What we use
Pivot	First element: <code>arr[low]</code>
Partition	Hoare partition
Left pointer	<code>i</code> moves from left toward right
Right pointer	<code>j</code> moves from right toward left
Stop condition	When <code>i >= j</code>

3. Partition — The Most Important Part

Take this array:

```
[4, 2, 7, 1, 3]
```

We call:

```
low = 0
high = 4
pivot = arr[low] = 4
```

Pointer setup:

```
i = low - 1 = -1
j = high + 1 = 5
```

The pointers are deliberately initialized outside the current range because the first operations are `i += 1` and `j -= 1`. Therefore, their first actual positions become `low` and `high`.

4. Why `low - 1` and `high + 1`?

`i = low - 1` followed by `i += 1` means `i` starts scanning at `low`.

j = high + 1 followed by **j -= 1** means j starts scanning at high.

```
i = low - 1
i += 1
→ i = low
```

```
j = high + 1
j -= 1
→ j = high
```

5. Pointer Rules

For pivot = 4:

i moves right while `arr[i] < pivot`.
When `arr[i] >= pivot`, i stops.

j moves left while `arr[j] > pivot`.
When `arr[j] <= pivot`, j stops.

When both stop, the elements at i and j are candidates for swapping. If `i < j`, swap them. If `i >= j`, partition is finished.

6. Complete Hand Trace

Starting array:

```
Index: 0 1 2 3 4
Array: [4, 2, 7, 1, 3]
      ↑ ↑
      low high
```

```
pivot = 4
i = -1
j = 5
```

Scan 1 — move i

```
i = 0
arr[0] = 4
4 >= 4 ✓
→ stop i at index 0
```

Scan 1 — move j

```
j = 4
arr[4] = 3
3 <= 4 ✓
→ stop j at index 4
```

Now `i < j`, so swap `arr[0]` and `arr[4]`:

```
[4, 2, 7, 1, 3]
 ↓ ↓
4 3
```

swap 4 and 3

```
[3, 2, 7, 1, 4]
```

Scan 2 — move i

```
i = 1
arr[1] = 2
2 < 4 ✓ → continue
```

```
i = 2
arr[2] = 7
7 >= 4 ✓ → stop
```

Scan 2 — move j

```
j = 3
arr[3] = 1
1 <= 4 ✓ → stop
```

Now $i < j$, so swap 7 and 1:

```
[3, 2, 7, 1, 4]
↑ ↑
i j
```

swap 7 and 1

```
[3, 2, 1, 7, 4]
```

Scan 3

```
Move i:
i = 3
arr[3] = 7
7 >= 4 ✓ → stop
```

```
Move j:
j = 2
arr[2] = 1
1 <= 4 ✓ → stop
```

```
Now:
i = 3
j = 2
```

```
i >= j → STOP PARTITIONING
```

7. What Did Partition Produce?

```
[3, 2, 1] | [7, 4]
left | right
↑
boundary
```

With Hoare partition, the returned value is a **partition boundary**. The pivot itself is not guaranteed to sit at the boundary. This is a critical distinction.

For this example, partition returns $j = 2$.

8. Now Quick Sort Recurses

```
quick_sort(arr, 0, 4)

p = partition(arr, 0, 4)
p = 2

quick_sort(arr, 0, 2)
quick_sort(arr, 3, 4)
```

So the problem becomes two smaller problems:

```
[3, 2, 1] | [7, 4]
0..2 3..4
```

Each smaller section is processed using the **same algorithm**.

9. The Complete Workflow

```
[4, 2, 7, 1, 3]
■
pivot = 4
```

```

■
partition
↓
[3, 2, 1] | [7, 4]
/ \
/ \
quick_sort quick_sort
0..2 3..4
■ ■
partition partition
↓ ↓
smaller smaller
parts parts
↓ ↓
base cases base cases
\ /
■■■■■ sorted ■■■

```

10. Complete Python Code

```

def partition(arr, low, high):

    pivot = arr[low]

    i = low - 1
    j = high + 1

    while True:

        while True:
            i += 1

        if arr[i] >= pivot:
            break

        while True:
            j -= 1

        if arr[j] <= pivot:
            break

    if i >= j:
        return j

    arr[i], arr[j] = arr[j], arr[i]

def quick_sort(arr, low, high):

    if low < high:

        p = partition(arr, low, high)

        quick_sort(arr, low, p)
        quick_sort(arr, p + 1, high)

arr = [4, 2, 7, 1, 3]

quick_sort(arr, 0, len(arr) - 1)

print(arr)

```

11. Understand the Two Functions

partition() has one job: rearrange the current range around the pivot and return the boundary.

quick_sort() has one job: repeatedly call **partition()** and recursively solve the two resulting ranges.

`partition()` → rearranges + returns boundary

`quick_sort()` → calls `partition()`

→ sorts left range

→ sorts right range

12. Base Case

`if low < high:`

If this condition is false, there is zero or one element in the current range. Such a range is already sorted, so recursion stops.

`low >= high`

↓

stop recursion

13. Quick Sort vs Merge Sort

Merge Sort	Quick Sort
Divide into halves	Choose pivot and partition
Merge step combines sorted halves	Partition rearranges elements
Needs extra array space	Can work in-place
Typical time: $O(n \log n)$	Average time: $O(n \log n)$
Worst case: $O(n \log n)$	Worst case: $O(n^2)$

14. Complexity

Average time: $O(n \log n)$ — partitions are reasonably balanced.

Worst-case time: $O(n^2)$ — can happen when partitions become very unbalanced, for example repeatedly choosing a poor pivot.

Auxiliary space: $O(\log n)$ on average for the recursion stack; $O(n)$ in the worst case. The partition itself is in-place.

15. Interview Answer

Quick Sort is a divide-and-conquer sorting algorithm. It chooses a pivot and partitions the array so that smaller elements go toward the left and larger elements go toward the right. It then recursively sorts the two partitions. With our implementation, the first element is the pivot and Hoare partitioning returns a boundary. Average time complexity is $O(n \log n)$, while worst-case time complexity is $O(n^2)$.

16. The Trick to Remember

QUICK SORT

1. Pick pivot
2. Move `i` from LEFT
3. Move `j` from RIGHT
4. Stop when each finds a wrong-side element
5. Swap
6. When `i >= j` → partition finished
7. Recursively `quick_sort(left)`
8. Recursively `quick_sort(right)`

Most important distinction: In Hoare partition, think *boundary*, not “pivot's final index.”